

Securing a Self-Hosted Navidrome Music Server

Technical Implementation Guide & Post-Mortem: Private HTTPS Access via Tailscale Serve

TARGET OS	APPLICATION	OVERLAY NETWORK	PROXY SERVICE
Debian Linux	Navidrome Media Server	Tailscale (Mesh VPN)	Tailscale Serve

1. OVERVIEW & PROJECT BACKGROUND

This technical report documents the configuration process for securing a self-hosted Navidrome music server on a Debian host. The objective was to transition from unencrypted HTTP connection strings to encrypted HTTPS using Tailscale's native web proxy infrastructure without exposing the application to the public Internet.

Core Objectives:

- Transport Layer Encryption:** Enable end-to-end `https://` traffic for music streaming and administration.
- Private Networking:** Restrict server access strictly to authorized devices within the private Tailnet mesh network.
- Simplified UX:** Replace numeric IP addresses and raw port numbers (e.g., `:4533`) with clean, valid domain endpoints on standard HTTPS port 443.

2. ORIGINAL CONFIGURATION

Prior to this deployment, Navidrome was fully installed and operational, bound to the local loopback interface (`127.0.0.1`) on default TCP port `4533`.

Original Endpoint Access Method:

```
http://<TAILSCALE_IP>:4533
```

Motivation for Change: Accessing web services over plain HTTP transmits session identifiers and authentication tokens in cleartext over local Wi-Fi and intermediary networks. Browsers also highlight unencrypted endpoints as "Not Secure." Rather than introducing complex third-party web servers or ACME certificate renewal scripts, Tailscale's integrated certificate and reverse proxy engine was selected.

3. TAILSCALE DNS & HOSTNAME ARCHITECTURE

To establish TLS certificates, a clear understanding of addressing layers within a mesh network is required:

Concept	Function / Scope	Example Identifier
Tailscale IP	Private IPv4/IPv6 address assigned to an individual mesh node.	<code>100.xxx.xxx.xxx / <TAILSCALE_IP></code>
MagicDNS / Tailnet DNS	Internal DNS routing hostnames directly to Tailscale IPs within the network.	<code><SERVER_NAME> . <TAILNET_NAME> . ts.net</code>
Public Internet DNS	Standard publicly resolvable global web domain records.	<code>example.com</code>

Note: The `.ts.net` domain namespace is managed by Tailscale. While the hostname is globally resolvable to internal Tailnet IP ranges, it does not make the underlying server accessible to the public Internet.

4. TAILNET RENAMING & PREPARATION

Prior to certificate generation, the global Tailnet namespace was updated via the Tailscale admin console to establish a clean naming standard for public documentation.

- Legacy Namespace Structure:** `<SERVER_NAME> . <OLD_TAILNET_NAME> . ts.net`
- Updated Namespace Structure:** `<SERVER_NAME> . <TAILNET_NAME> . ts.net`

5. ENABLING HTTPS CERTIFICATES

Tailscale provides automated provisioning of publicly trusted TLS/SSL certificates issued by **Let's Encrypt** for nodes operating inside a Tailnet.

Administrative Steps Taken:

1. Accessed the **Tailscale Admin Console** → **DNS** tab.
2. Scrolled down to **HTTPS Certificates** and toggled the feature to **Enabled**.

This setting authorizes Tailscale nodes to request machine-specific TLS certificates for their respective `<SERVER_NAME>.<TAILNET_NAME>.ts.net` domain name.

6. SECURITY EVALUATION: PRIVACY VS. EXPOSURE

Before proceeding with proxy implementation, a critical security evaluation was conducted regarding public certificate issuance.

Security Distinction: Certificate Visibility vs. Network Access

Certificate Transparency (CT) Logs: Every public certificate issued by Let's Encrypt is recorded in a publicly searchable CT log. The hostname `<SERVER_NAME>.<TAILNET_NAME>.ts.net` is visible publicly. However, CT logs only reveal that a domain name exists; they do **not** grant public network access to the server behind it.

Service Comparison:

Feature	Traffic Pathway	Public Internet Access?
Tailscale Serve	Proxies requests internally within the private encrypted mesh network.	No (Strictly Private)
Tailscale Funnel	Routes public Internet traffic through Tailscale edge servers to local nodes.	Yes (Publicly Exposed)

This implementation strictly utilizes **Tailscale Serve**, ensuring zero public exposure.

7. PRE-DEPLOYMENT HEALTH VERIFICATION

Before altering network pathways, Navidrome's local listener was verified on the host machine using `curl`.

```
curl http://127.0.0.1:4533
```

Response Received:

```
<a href="/app/">Found</a>.
```

This HTTP 302 redirect confirmed that the local Navidrome process was active and serving requests on loopback.

Troubleshooting Note: An initial mistyped command (`curl http://127/0/0/1:4533`) failed due to invalid IP formatting. Loopback strictly requires `127.0.0.1`.

8. TAILSCALE SERVE DEPLOYMENT & INTERACTIVE LESSONS

Tailscale Serve was initially executed in foreground mode to verify proxy behavior:

```
sudo tailscale serve 4533
```

Interactive Output Displayed:

```
Available within your tailnet:

https://<TAILNET_HOSTNAME>/
|-- proxy http://127.0.0.1:4533
```

9. FINAL BACKGROUND SERVICE CONFIGURATION

To ensure the HTTPS reverse proxy remains permanently active across terminal disconnects and system reboots, the Serve configuration was deployed using the background flag:

```
sudo tailscale serve --bg 4533
```

Verification Steps:

1. Check active proxy status:

```
tailscale serve status
```

Active Response:

```
https://<TAILNET_HOSTNAME> (Tailscale TLS cert)
|-- / proxy http://127.0.0.1:4533
```

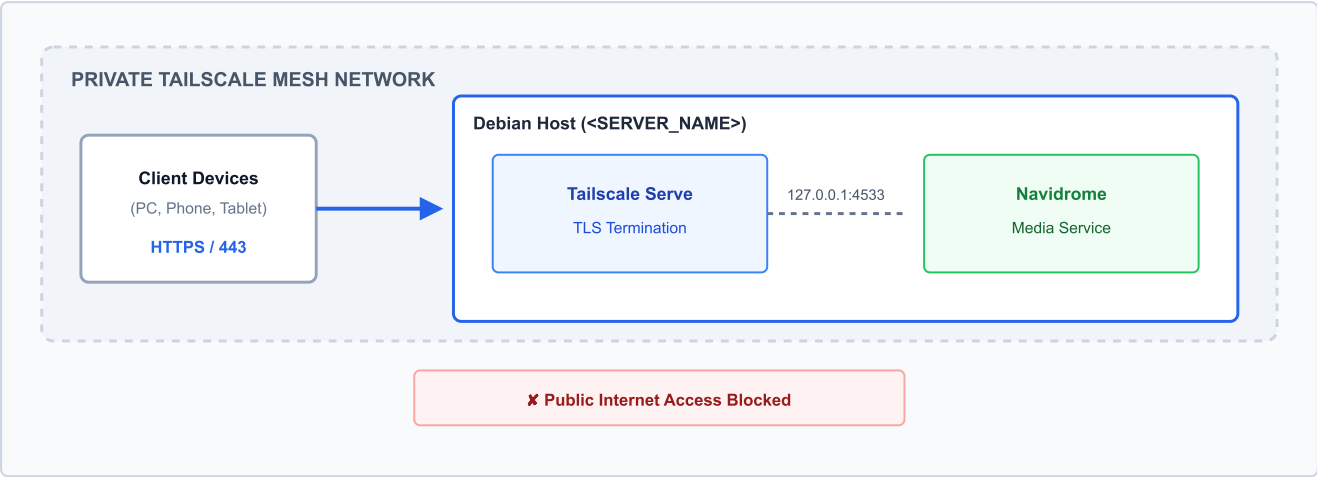
2. Verify client access: On any secondary device authenticated to the Tailnet, open a web browser and navigate directly to:

```
https://<TAILNET_HOSTNAME>/
```

Client devices connect securely over port 443 with a valid browser trust lock. No custom port specifications or local client proxy rules are required.

10. FINAL SYSTEM ARCHITECTURE

The resulting architecture maintains a clear separation between the public Internet and the private encrypted mesh network:



11. FINAL STATE COMPARISON & SECURITY SUMMARY

Parameter	Initial State	Final State
Access Protocol	Unencrypted HTTP (http://)	Encrypted HTTPS (https://)
Address Endpoint	http://<TAILSCALE_IP>:4533	https://<TAILNET_HOSTNAME>/
Transport Security	Cleartext traffic on network	TLS 1.3 Encrypted via Let's Encrypt
Port Overhead	Manual application port (4533)	Standard Web HTTPS (443)
Network Boundaries	Private Mesh Network	Private Mesh Network (Unchanged)